Four-Character Block Transposition

Technical documentation of a C++ console program | Version 1.0

TECHNICAL NOTE A review of the supplied source file and the author's preliminary write-up. The source code is documented as written; no changes have been made to its behavior.

1. Purpose and scope

This command-line program reads one line of text, changes the order of characters in consecutive four-character blocks, and prints the transformed result. The same operation can be applied a second time to restore the original byte sequence. The implementation is concise, works in place, and uses only the C++ standard library.

TECHNICAL NOTE Although the preliminary write-up refers to the Julius Caesar cipher, the actual program implements a fixed transposition of character positions. Caesar encryption replaces each alphabetic symbol by a shifted symbol; this program does not shift letters, use a key, or perform alphabetic substitution.

2. Functional requirements and I/O

Item	Implemented behavior
Input	One complete line from standard input, including spaces (std::getline).
Output	The reordered text, followed by a newline, on standard output.
Processing unit	Contiguous groups of up to four std::string elements (bytes).
State	One mutable std::string; the input is modified in place.
User interface	Console prompt: Enter the text:
Parameters	No encryption key, alphabet, shift amount, or file input.

3. Algorithm design

Let a, b, c, and d be the elements of one full block. The algorithm exchanges elements at offsets 0 and 2, then exchanges offsets 1 and 3. For the final, incomplete block, only exchanges whose positions exist are performed.

Final block length	Transformation	Example
4	(a, b, c, d) → (c, d, a, b)	ABCD → CDAB
3	(a, b, c) → (c, b, a)	ABC → CBA
2	No change	AB → AB
1	No change	A → A
0	No processing	empty → empty

4. How the source code works

Source element	Responsibility
#include <iostream>	Provides std::cin and std::cout for console input and output.
#include <string>	Provides std::string and std::getline.

Source element	Responsibility
#include <utility>	Provides std::swap, used to exchange two elements.
#include <cstddef>	Provides std::size_t, the unsigned index type.
std::getline(std::cin, text)	Reads a full input line; the newline delimiter is not stored.
for (...; i + 2 < text.size(); i += 4)	Visits each four-element block, provided it has at least three elements.
std::swap(text[i], text[i + 2])	Exchanges positions 1 and 3 of the current block.
if (i + 3 < text.size())	Checks that position 4 exists before exchanging positions 2 and 4.
std::cout << text << '\n'	Prints the transformed line and ends it with a newline.

5. Execution trace

Consider input ABCDEFG. Its length is 7; the loop executes at i = 0 and i = 4.

Iteration	Operation	Current text
Start	Read complete line	ABCDEFG
i = 0	Swap A ↔ C; then B ↔ D	CDABEFG
i = 4	Swap E ↔ G; position 4 of this block does not exist	CDABGFE
Finish	Print the result	CDABGFE

Pseudocode:

```
read an entire line into text
for i = 0; i + 2 < text.length; i = i + 4:
    swap(text[i], text[i + 2])
    if i + 3 < text.length:
        swap(text[i + 1], text[i + 3])
print text
```

6. Properties and correctness

- Length preservation: each operation is a swap, so no elements are added or removed.
- Data preservation: the output contains exactly the same bytes, although their positions can change.
- Reversibility: each swap reverses itself; executing the full transformation twice produces the original input (T(T(x)) = x).
- Termination: the loop index increases by four each iteration, while the string length remains fixed.
- No out-of-range access: the loop condition guarantees i + 2 exists, and the inner condition guards i + 3.

7. Complexity analysis

For an input of n bytes, the loop visits at most ⌊ n/4⌋ complete blocks plus one final three-byte block. Each iteration performs no more than two constant-time swaps. Therefore, the processing time is O(n), and the algorithm uses O(1) auxiliary memory beyond the input string. Storing the input itself requires O(n) space.

8. Test cases and observed results

The following outputs were verified against the provided C++ source after compilation with g++ and the C++11 language standard.

Input	Expected / observed output	Reason
[empty line]	[empty line]	Zero elements
A	A	One-element suffix
AB	AB	Two-element suffix
ABC	CBA	Three-element exchange
ABCD	CDAB	One full block
ABCDE	CDABE	Full block + one leftover
ABCDEF	CDABEF	Full block + two leftover
ABCDEFG	CDABGFE	Full block + three leftover
ABCDEFGH	CDABGHEF	Two full blocks
HELLO WORLD	LLHEWOO DLR	Whitespace moves too

9. Build and run

In Code::Blocks, create or open a C++ console project, replace the source with the supplied main.cpp content, then choose Build and Run (commonly F9). Use a toolchain with C++11 support. To compile manually using GNU g++ in a shell:

```
g++ -std=c++11 -Wall -Wextra -pedantic "main(1).cpp" -o transform
./transform
```

On Windows using a MinGW-based g++ toolchain, change the output name to transform.exe and run it from the appropriate terminal. The program waits for a single line after showing the prompt.

10. Limitations and security

- Not Caesar cipher or secure encryption: the pattern is public and fixed. This is a learning exercise in indexing, loops, bounds checks, and transposition, not a method for protecting secrets.
- Character encoding: std::string indexes bytes. Reordering bytes can split multibyte UTF-8 characters (including Persian letters), producing invalid or unreadable text. Examples in this document use ASCII.
- Single-line input: std::getline stops at a newline and the program does not process later lines.
- No explicit input-error handling: if the input stream is unavailable, the string remains empty and an empty line is printed.

11. Corrections to the preliminary write-up

Preliminary claim	Accurate statement
"Julius Caesar Cipher"	A fixed four-position character transposition; no alphabet shift.
"get number from the user"	Read a string (one complete line) from standard input.
"place it on a array"	Store the input in a std::string and modify indexed positions.
"A swap with D"	For ABCD, A swaps with C and B swaps with D.
"use 4 library"	Four standard library headers are included; each serves a specific purpose.

12. Potential extensions

- Extract the transformation into a reusable function, e.g. std::string transposeBlocks(std::string text).

- Add explicit tests for input lengths 0–8, punctuation, and round-trip identity.
- For Unicode text, operate on validated Unicode code points or grapheme clusters rather than raw UTF-8 bytes.
- If the original goal is Caesar cipher, implement a separate alphabet-aware shift algorithm with a user-supplied key.

Appendix A. Exact supplied source code

Reproduced without modifying the algorithm (only surrounding blank lines are omitted).

```
#include <iostream>
#include <string>
#include <utility>
#include <cstddef>

int main()
{
    std::string text;

    // Read an entire line of input, including spaces.
    std::cout << "Enter the text: ";
    std::getline(std::cin, text);

    // Process the text in blocks of four characters.
    for (std::size_t i = 0; i + 2 < text.size(); i += 4)
    {
        // Swap the first and third characters of the current block.
        std::swap(text[i], text[i + 2]);

        // Swap the second and fourth characters, if both exist.
        if (i + 3 < text.size())
        {
            std::swap(text[i + 1], text[i + 3]);
        }
    }

    // Display the transformed text.
    std::cout << text << '\n';

    return 0;
}
```